Classification supervisée

Construire un modèle : choix, évaluation, reproductibilité

Université de Montpellier – LIRMM

HAI817I

Pascal Poncelet
Pascal.Poncelet@lirmm.fr
http://www.lirmm.fr/~poncelet

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Du tableau au modèle de classification

- Nous avons appris à rendre une table exploitable.
- Objectif maintenant : construire un modèle de classification fiable.
- Étapes clés :
  1. définir la cible et les variables disponibles au moment de prédire
  2. définir le pré-traitement, sans fuite de données
  3. choisir une baseline, puis un premier modèle
  4. choisir des métriques et une méthode d'évaluation
  5. comparer, ajuster, puis figer un pipeline reproductible

Ingénierie et classification : une boucle vertueuse

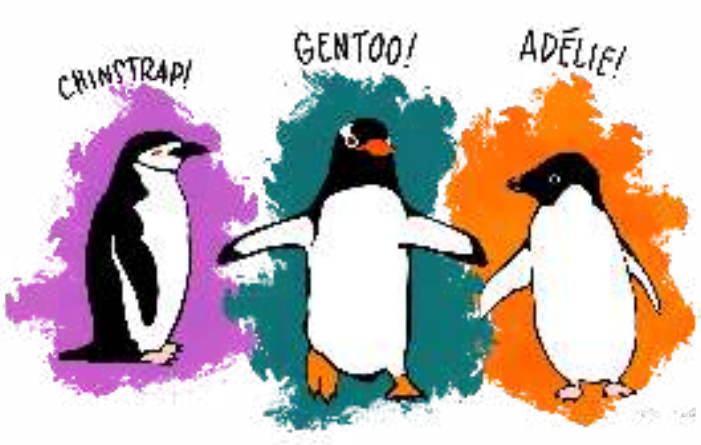
- On commence par une première phase d'ingénierie des données.
- Ensuite, on cherche un premier modèle simple.
- Les résultats du modèle servent de diagnostic :
  - variables peu utiles ou redondantes,
  - erreurs typiques et cas difficiles,
  - besoin de nouvelles variables ou d'un autre pré-traitement.
- On améliore, on réévalue, et on garde une démarche reproductible.

Formuler un problème de classification

- On dispose d'exemples décrits par des variables (features).
- Chaque exemple a une étiquette à prédire (classe).
- On cherche une règle de décision qui généralise à de nouveaux exemples.
- Attention : être sûr de ce qui est disponible au moment de prédire.

Jeu de données fil conducteur : Palmer Penguins

- `Palmer Penguins`.
- 344 manchots, 3 espèces (Adelie, Chinstrap, Gentoo).
- Variables : mesures (bec, nageoire, masse) et contexte (île, sexe, année).
- Cible : prédire l'espèce à partir des variables disponibles.



Source : Allison Horst

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Ce que l'on fixe avant de comparer des modèles

- La cible : la classe à prédire.
- Les variables : ce qui sera disponible au moment de prédire.
- Une baseline : un point de comparaison minimal.
- Un protocole d'évaluation défini avant d'optimiser.

Définir X et y

- X : les variables explicatives (une matrice).
- y : la cible (un vecteur, la classe à prédire).
- Exemple (Penguins) :

```
y = df["species"]
X = df.drop(columns=["species"])
```

Pourquoi une baseline ?

- Sans baseline, un score n'a pas de sens.
- Une baseline simple répond à une question :

  Est-ce que le modèle fait mieux qu'une règle naïve ?
- Elle permet d'interpréter les scores par comparaison.

Baseline scikit-learn : règle naïve

- Baseline utilisable comme un classifieur standard :

```
from sklearn.dummy import DummyClassifier
baseline = DummyClassifier(strategy="most_frequent",
random_state=0)
```

- Idée : prédire toujours la classe la plus fréquente.
- Même protocole d'évaluation que pour les autres modèles.
- On peut aussi choisir d'autres stratégies pour définir la règle naïve.

Entraîner puis prédire

- Tous les classifieurs scikit-learn suivent le même principe.
- On entraîne avec `fit`, puis on prédit avec `predict` sur d'autres exemples.
- Exemple :

```
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

Un protocole simple : séparer pour évaluer

- On met de côté une partie des données pour tester.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)
```

- On entraîne sur `X_train` (80%) et on évalue sur `X_test` (20%).

Réserver un test final (si on a assez de données)

- Idée : garder un petit jeu jamais utilisé pendant les choix, comme des données nouvelles.

```
X_train, X_test_final, y_train, y_test_final =
train_test_split(
    X, y, test_size=0.1, stratify=y, random_state=0)
```

- Ensuite, on travaille sur `X_train` et `y_train` que l'on renomme `X` et `y` ☺.
- `X_test_final` sert à une seule vérification finale ☺ (c'est rassurant de voir que ça fonctionne sur des données jamais vues).
- Avec peu de données, on ne peut pas toujours se le permettre.

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Hypothèse : entrée du classifieur après `PenguinsPreprocessor`

- On suppose qu'après `PenguinsPreprocessor`, toutes les colonnes sont numériques.

Colonne en entrée	Type	Exemple de valeur
`Culmen Length (mm)`	numérique	45.2
`Culmen Depth (mm)`	numérique	17.1
`Flipper Length (mm)`	numérique	210
`BodyMass_Cat_*`	discrétisé et OneHotEncoding	0 ou 1
`Island_*`	binaire (0/1)	0 ou 1
`Sex_*`	binaire (0/1)	0 ou 1

Séparer pour évaluer

- On met de côté une partie des données pour mesurer la qualité.
- Rappel : si un `X_test_final` existe, on travaille sur le reste, noté `X` et `y`.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)
```

Normalisation : rappel

- Sensibles à l'échelle : kNN, SVM, régression logistique.
- Moins sensibles : arbres de décision, forêts.
- On standardise uniquement les mesures (pas les colonnes 0/1 du one-hot).

```
from sklearn.preprocessing import StandardScaler

num_cols = ["Culmen Length (mm)","Culmen Depth
(mm)","Flipper Length (mm)"]
scaler = StandardScaler()
X_train[num_cols] =
scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

- On apprend sur `X_train` : `X_test` doit rester comme des données nouvelles !

Entraîner et prédire

- On entraîne sur `X_train` et on prédit sur `X_test`.

```
baseline.fit(X_train, y_train)
y_pred = baseline.predict(X_test)
```

- `y_pred` contient une espèce prédite pour chaque manchot du test.

Évaluer : une idée simple

- On compare `y_pred` (prédit) à `y_test` (réel).
- Deux questions :

  combien de prédictions sont correctes ?
  quelles erreurs sont faites, et sur quelles classes ?

Matrice de confusion : l'objet de base

- Une matrice qui compare classes réelles et classes prédites.
- Lignes : classe réelle.
- Colonnes : classe prédite.
- Diagonale : bonnes prédictions.

Construire la matrice de confusion

- À partir de `y_test` et `y_pred` :

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
```

Mini-exemple (3 classes)

Réel \ Prédit	Adelie	Chinstrap	Gentoo
Adelie	18	2	0
Chinstrap	3	9	1
Gentoo	0	1	16

- Bons classements : $18 + 9 + 16 = 43$ (diagonale).
- Total : $(18 + 2 + 0) + (3 + 9 + 1) + (0 + 1 + 16) = 50$.
- Exemples d'erreurs : 2 Adelie prédits Chinstrap, 3 Chinstrap prédits Adelie.

Accuracy : proportion de bonnes prédictions

- Définition : bonnes prédictions divisé par le total.
- Sur l'exemple :

  bonnes prédictions = 43
  total = 50
  accuracy = 43/50 = 0.86
- Attention : si les classes sont déséquilibrées, l'accuracy peut être trompeuse.

```
from sklearn.metrics import accuracy_score

acc = accuracy_score(y_test, y_pred)
```

TP, FP, FN, TN : cas binaire

- Quand on évalue une classe, on raisonne en binaire : `classe` ou `non classe`.

Réel \ Prédit	classe	non classe
classe	TP	FN
non classe	FP	TN

TP : classe réelle et prédite classe

FP : non classe réelle mais prédite classe

FN : classe réelle mais prédite non classe

TN : non classe réelle et prédite non classe

- En multi-classes, on choisit une classe et on regroupe toutes les autres en `non classe`.

Réduire à une classe : TP, FP, FN, TN

- Exemple : on fixe la classe `Adelie`.
- On regroupe toutes les autres espèces dans `non Adelie`.

Réel \ Prédit	Adelie	non Adelie
Adelie	TP	FN
non Adelie	FP	TN

TP : Adelie réel et prédit Adelie

FP : non Adelie réel mais prédit Adelie

FN : Adelie réel mais prédit non Adelie

TN : non Adelie réel et prédit non Adelie

Métriques : précision, rappel, F1

- Précision : parmi les prédits Adelie, quelle part est correcte ?

$$\text{Précision} = \frac{TP}{TP + FP}$$

- Rappel : parmi les Adelie réels, quelle part est retrouvée ?

$$\text{Rappel} = \frac{TP}{TP + FN}$$

- F1 : compromis entre précision et rappel.

$$F1 = \frac{2 \times \text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Classification report : résumé par classe

- Le report calcule précision, rappel et F1 pour chaque classe.
- Il indique aussi le nombre d'exemples par classe (support).

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Rapport de classification : un résumé utile

- Le rapport donne, pour chaque classe :
  - précision (precision)
  - rappel (recall)
  - F1-score
  - support (nombre d'exemples)
- Rappel :
  - précision : qualité des prédictions de la classe
  - rappel : capacité à retrouver la classe

Rapport de classification (équilibré) :

	precision	recall	f1-score	support
0	0.91	0.85	0.88	301
1	0.86	0.92	0.89	299
accuracy			0.88	600
macro avg	0.88	0.88	0.88	600
weighted avg	0.88	0.88	0.88	600

Rapport de classification (léger déséquilibre) :

	precision	recall	f1-score	support
0	0.93	0.94	0.94	419
1	0.85	0.85	0.85	181
accuracy			0.91	600
macro avg	0.89	0.89	0.89	600
weighted avg	0.91	0.91	0.91	600

Rapport de classification (fort déséquilibre) :

	precision	recall	f1-score	support
0	0.98	1.00	0.99	1466
1	0.00	0.00	0.00	34
accuracy			0.98	1500
macro avg	0.49	0.50	0.49	1500
weighted avg	0.96	0.98	0.97	1500

Macro avg et weighted avg : pourquoi c'est important

- macro avg : moyenne simple sur les classes
  chaque classe compte autant, même si elle est rare
- weighted avg : moyenne pondérée par le support
  une classe majoritaire pèse beaucoup plus
- Déséquilibre (figure) :
  accuracy très élevée (0.98) alors qu'une classe est ignorée
  macro avg chute (0.49), weighted avg reste élevé (0.97)
- Réflexe : regarder macro avg et les scores par classe.

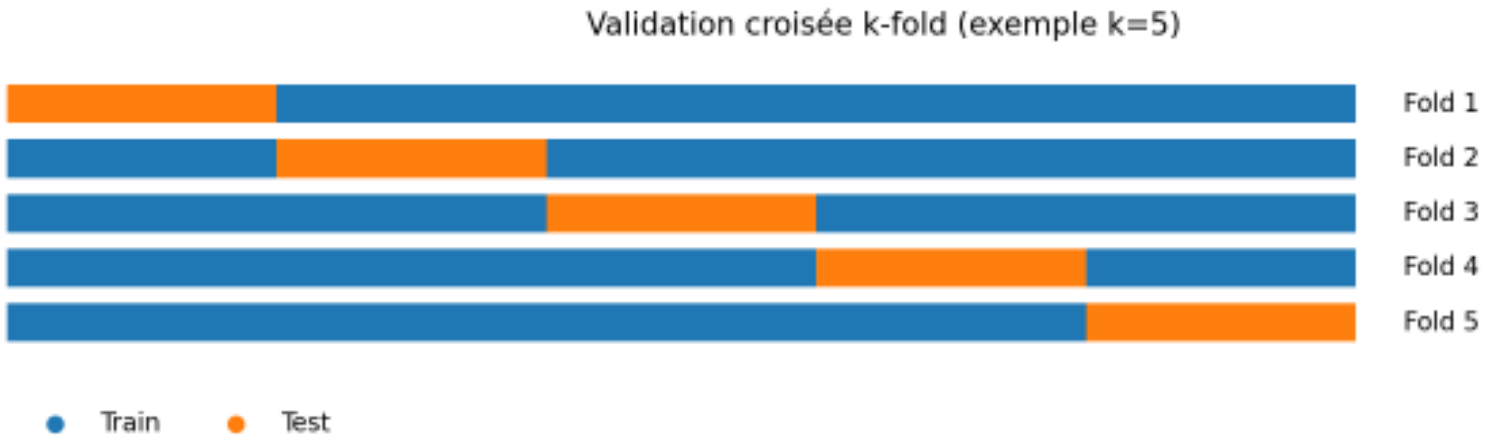
Un seul test : est-ce vraiment représentatif ?

- On a évalué sur un seul jeu de test, issu d'un seul découpage.
- Ce jeu peut ne pas représenter toutes les situations des données.
- Un autre découpage peut conduire à d'autres erreurs et d'autres scores.
- Il est donc indispensable d'évaluer sur plusieurs découpages.

Validation croisée k-fold : principe

- On découpe les données en k blocs.
- On répète k fois :

  on entraîne sur $k-1$ blocs,
  on teste sur le bloc restant.
- On obtient k scores, plus représentatifs qu'un seul découpage.



k-fold en scikit-learn : scores, moyenne, dispersion

- Exemple avec la baseline :

```
from sklearn.model_selection import StratifiedKFold,
cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=0)
scores = cross_val_score(baseline, X, y, cv=cv,
scoring="accuracy")
```

- `scores` contient les 5 valeurs.
- Résumé : `scores.mean()` et `scores.std()`.
- Interprétation : moyenne (niveau) et écart-type (stabilité selon le découpage).

k-fold et normalisation

- Si on normalise avant le k-fold, on utilise aussi des informations des folds de test.
- Il faut normaliser à l'intérieur de chaque fold :

  apprendre la normalisation sur la partie train du fold,
  l'appliquer à la partie test du fold.
- Sinon l'évaluation devient trop optimiste.

k-fold avec normalisation

- Exemple sur l'accuracy :

```
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=0)
scores = []
for tr, te in cv.split(X, y):
    Xtr, Xte = X.iloc[tr].copy(), X.iloc[te].copy()
    ytr, yte = y.iloc[tr], y.iloc[te]
    scaler = StandardScaler()
    Xtr[num_cols] = scaler.fit_transform(Xtr[num_cols])
    Xte[num_cols] = scaler.transform(Xte[num_cols])
    baseline.fit(Xtr, ytr)
    ypred = baseline.predict(Xte)
    scores.append(accuracy_score(yte, ypred))
scores = np.array(scores)
```

Bilan

- On a un point de départ : la baseline.
- On sait évaluer avec des métriques et comprendre les erreurs.
- On sait rendre l'évaluation plus fiable avec le k-fold.
- Prochaine étape : comparer plusieurs classifieurs avec le même protocole.

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Comparer plusieurs modèles

- Objectif : tester plusieurs classifieurs avec leurs réglages par défaut.
- On choisit des familles différentes :

  modèles linéaires (LR),
  voisinage (k-NN),
  méthodes à marge (SVM),
  arbres et ensembles : Random Forest, Gradient Boosting.
- Cela donne vite une première hiérarchie.
- Biais : les valeurs par défaut ne sont pas neutres, mais elles évitent de lancer des batteries d'essais pour tous les modèles (merci pour le CO_2 ☺).
- La comparaison sert surtout à sélectionner quelques candidats.

Astuce : visualiser les données avant de comparer

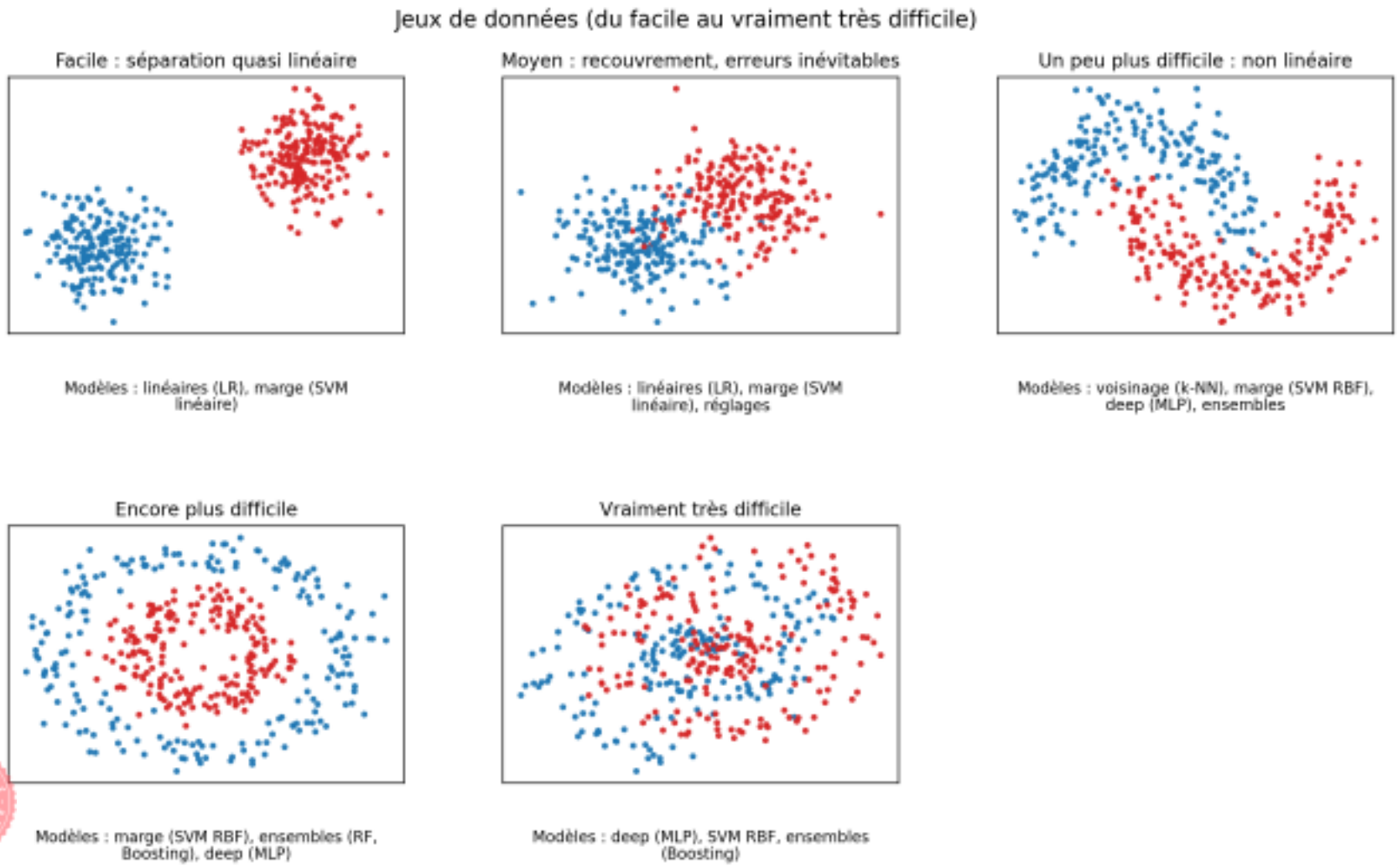
- Une projection 2D ou 3D aide à comprendre la séparabilité des classes.
- Cela peut aider à savoir quel modèle utiliser.
- Outils courants :

  ACP : rapide, linéaire, bon point de départ
  t-SNE : met l'accent sur les voisinages, plus coûteux
  UMAP : voisinages, souvent plus stable que t-SNE

- On projette puis on colore par espèce.

Exemples de séparabilité (de facile à vraiment difficile)

- Objectif : relier la géométrie des données aux familles de modèles possibles.



Frontières de décision

- Une même tâche peut demander une frontière linéaire ou très non linéaire.

Tester plusieurs classifieurs : pseudo-code

- Même protocole pour tous les modèles (mêmes données, même k-fold).
- Définir les modèles :

```
models = {
  "SVM": SVC(...),
...
  "NB": GaussianNB(...)
}
```

- Fixer le k-fold :

```
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=0)
```

- Boucler et stocker les scores :

```
results = {}
for name, clf in models.items():
    scores = cross_val_score(clf, X, y, cv=cv,
scoring="accuracy")
    results[name] = scores
```

Quid des modèles qui ont besoin de normalisation ?

- Rappel : certains classifieurs sont sensibles à l'échelle des variables.
- Dans notre jeu Penguins, plusieurs variables sont numériques :

```
num_cols = ["Culmen Length (mm)","Culmen Depth
(mm)","Flipper Length (mm)"]
```

- En k-fold, la règle est toujours la même :

  apprendre la normalisation sur `X_train` du fold,
  appliquer la transformation sur `X_test` du fold.

Pipeline : regrouper pré-traitement et classifieur

- Un pipeline enchaîne des étapes comme un seul objet.
- `clf` désigne un classifieur (SVM, k-NN, arbre, etc.).
- `preprocess` désigne un pré-traitement appliqué avant le modèle.
- Exemple (illustration) :

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

preprocess = StandardScaler()
model = Pipeline([("scaler", preprocess), ("clf", clf)])
```

- Pour Penguins, on standardise uniquement `num_cols` (pas les colonnes 0/1).

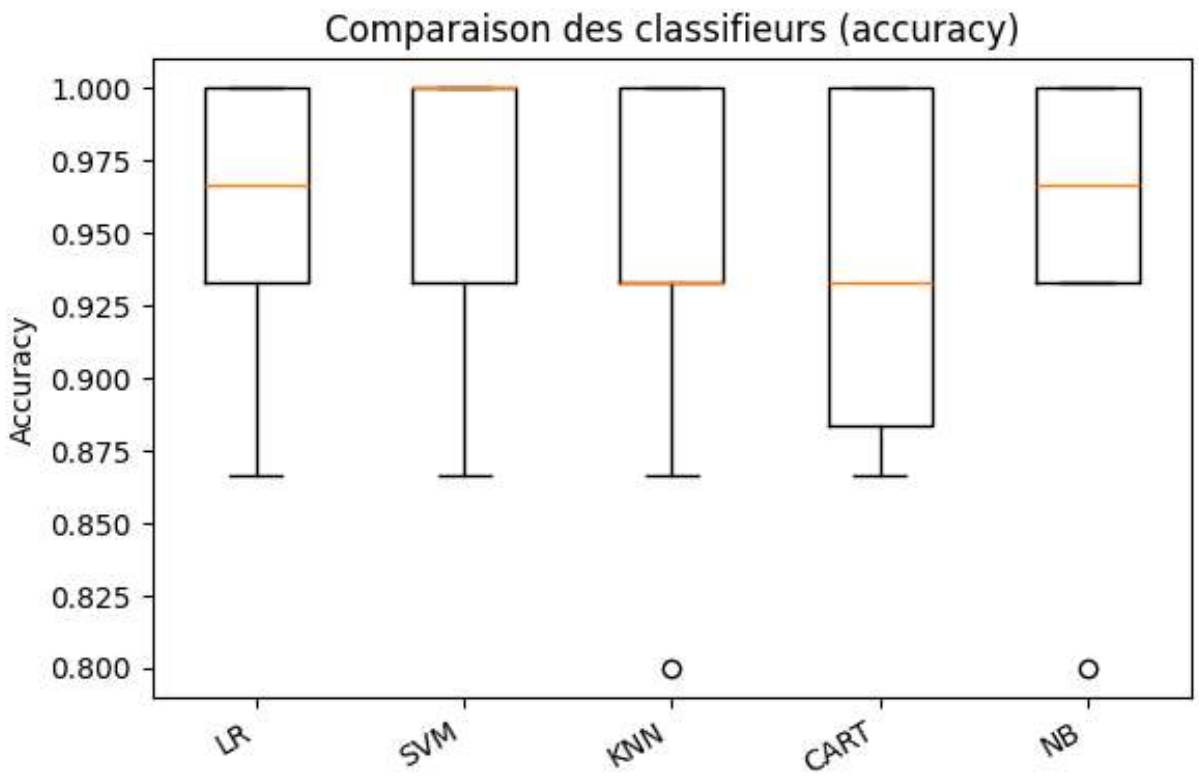
Tester plusieurs classifieurs : intégrer un pipeline

- Objectif : garder la même boucle, mais évaluer un objet unique pré-traitement puis classifieur.
- Dans la boucle, remplacer `clf` par `model` :

```
needs_scaling = {"SVM": True, "kNN": True, "LR": True,
"MLP": True}
results = {}
for name, clf in models.items():
    if needs_scaling.get(name, False):
      preprocess = StandardScaler()
      model = Pipeline([("scaler", preprocess), ("clf",
clf)])
    else:
      model = clf
    scores = cross_val_score(model, X, y, cv=cv,
scoring="accuracy")
    results[name] = scores
```

Comparer plusieurs modèles : boxplots des scores k-fold

- Chaque boîte résume les scores obtenus sur les différents folds.
- Lecture :
  médiane : performance typique,
  dispersion : stabilité selon le découpage,
- Objectif : repérer quelques candidats à analyser et à régler.



Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Pourquoi chercher des hyperparamètres ?

- Les réglages par défaut donnent un premier repère, pas le meilleur modèle.
- Deux ou trois paramètres peuvent changer fortement le résultat.
- Objectif : améliorer un candidat, sans tricher sur l'évaluation.

Exemples d'hyperparamètres

- Régression logistique :

  `C` : force de régularisation
  `penalty` : type de régularisation

- SVM :

  `C` : compromis entre marge et erreurs
  `kernel`, `gamma` : frontière linéaire ou non linéaire

- k-NN :

  `n_neighbors` : nombre de voisins
  `weights` : vote uniforme ou pondéré

- Arbres / ensembles :

  `max_depth`, `min_samples_leaf` : complexité
  `n_estimators`, `learning_rate` (boosting) : nombre et poids des arbres

Recherche simple : GridSearchCV

- On définit une grille de valeurs à tester.
- Chaque combinaison est évaluée par validation croisée.
- Résultat : le meilleur couple (modèle, réglages) selon une métrique.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(random_state=0)

param_grid = {
  "n_estimators": [100, 300, 600],
  "max_depth": [None, 5, 10],
  "min_samples_leaf": [1, 2, 5]
}

search = GridSearchCV(model, param_grid, cv=5,
scoring="accuracy")
search.fit(X, y)
```

- Résultat : `search.best_params_` et `search.best_score_`.

RandomizedSearchCV : quand la grille explose

- Quand il y a trop de combinaisons, on échantillonne au hasard.
- Intérêt : tester plus large avec un budget fixé (nombre d'essais).

```
search = RandomizedSearchCV(model, param_grid,
n_iter=50, cv=5)
```

- Bon réflexe : commencer large, puis affiner autour des meilleurs.

Optuna : optimiser avec un budget

- Même idée : explorer l'espace des hyperparamètres, mais de façon adaptative.
- On fixe un budget (nombre d'essais) et une fonction objectif.
- Intérêt :

  explore plus intelligemment qu'une grille,
  s'arrête tôt sur les réglages mauvais,
  garde un historique complet des essais.
- En pratique : on évalue chaque essai avec le même k-fold.

Conclusion : une recherche efficace, pas exhaustive

- On peut chercher en même temps la meilleure famille de modèles et les meilleurs hyperparamètres. Mais cela devient vite très coûteux : beaucoup d'essais, beaucoup de calcul, beaucoup de temps.
- Réflexe : pré-sélectionner quelques candidats, puis optimiser seulement ceux-là.
- À ce stade, on doit avoir :

  un protocole d'évaluation stable,
  des pré-traitements retenus,
  un modèle final et ses réglages.
- Prochaine étape : figer un pipeline reproductible pour une mise en production.

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Passer en production : figer une solution

- Les expérimentations ont permis de retenir :
  - un meilleur modèle et ses hyperparamètres,
  - les pré-traitements à appliquer.
- Objectif maintenant : construire un objet unique, reproductible, réutilisable.

Dernière vérification : test final (si disponible)

- Si on a réservé un jeu `X_final`, c'est le moment de l'utiliser.
- Étapes :

  ré-entraîner sur toutes les données sauf `X_final`,
  tester une seule fois sur `X_final`.
- Intérêt : vérifier sur des données jamais utilisées pendant les choix.

Pré-traitements : une seule classe, une seule logique

- On regroupe tous les pré-traitements retenus dans `PenguinsPreprocessor`.
- La classe doit respecter `fit/transform` :

  `fit` apprend uniquement ce qui dépend des données (ex. encodage),
  `transform` applique la même transformation aux nouvelles données.
- Objectif : éviter les écarts entre entraînement et utilisation réelle.

Pipeline final : pré-traitement puis classifieur

- On construit un pipeline unique pour l'usage réel.

```
prep = PenguinsPreprocessor(...)
clf = ...   (meilleur modèle + hyperparamètres)
final_model = Pipeline([("prep", prep), ("clf", clf)])
```

- On entraîne sur toutes les données disponibles :

```
final_model.fit(X, y)
```

Clin d'oeil : `PenguinsPreprocessor` version production

- Dans le cours d'ingénierie, `PenguinsPreprocessor` était une base de travail.
- En production, elle doit refléter exactement les choix retenus.
- Exemple concret : `Body Mass (g)`

  on a discrétisé en `BodyMass_Cat` (léger / lourd),
  donc il faut aussi l'encoder avec un `OneHotEncoder` ☺

- Colonnes à exclure : ne pas oublier que ce n'est pas seulement `Comments` ☺

  ex. `studyName`, `Sample Number`, `Individual ID`, `Date Egg`,
  ex. `Delta 15 N (o/oo)`, `Delta 13 C (o/oo)`.

- Même logique pour les manquants, les encodages et la normalisation des numériques.

Sauvegarder et recharger le modèle

- Sauvegarde :

```
import joblib
joblib.dump(final_model, "penguins_model.joblib")
```

- Chargement et prédiction sur de nouvelles données :

```
model = joblib.load("penguins_model.joblib")
y_new = model.predict(X_new)
```

- Le pipeline applique automatiquement les mêmes pré-traitements qu'à l'entraînement.

Plan

1. Introduction
2. Cadrer le problème de classification
3. Premier modèle : entraîner et évaluer
4. Comparer plusieurs modèles
5. Chercher de bons hyperparamètres
6. Mise en production
7. Conclusion

Conclusion

- On part d'une table et on construit une solution de classification fiable.
- Les points clés :

  éviter les fuites de données (pré-traitements appris uniquement sur le train),
  évaluer de façon stable (k-fold, pas un seul découpage),
  comparer des familles de modèles, puis optimiser les hyperparamètres,
  valider une dernière fois sur un test final si disponible.
- Pour terminer : figer un pipeline complet (pré-traitements + modèle), sauvegarder, recharger, prédire sur de nouvelles données.

Des questions ?